

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	T Jyothi Reddy	Reg. No.:	192472079
Section / Batch:	2024	Date:	

Code of Conduct

I T Jyothi Reddy(192472079) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T Jyothi Reddy

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select $N = 1, 2$, or 3 , display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights. The system should generate the top-5 next-word predictions using:

- Unsmoothed N-gram model
- Backoff model
- Deleted Interpolation model

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat."

and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- Unigram model
- Bigram model
- Trigram model
- Smoothed model

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- Word/tag frequencies
- Tag transition probabilities
- Probabilities obtained from a training corpus

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1	4	4	3	3	4		92
2	4	4	3	3	4		
3	3	3	4	4	4		
4	4	4	3	3	4		

Faculty In-charge	Course Coordinator	Program Director
Signature: Dr. Kumaragurubaran.T _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Task 1 – N-gram Based Text Prediction

What this program does

- Preprocesses an English corpus.
- Performs sentence segmentation and tokenization.
- Builds **Unigram, Bigram and Trigram** models.
- Calculates frequency and Maximum Likelihood Probability.
- Accepts N = 1, 2, or 3.
- Predicts the **top-5 next words**.
- Demonstrates an unseen N-gram.
- Tests multiple sentences and calculates prediction accuracy.

Code:

```
import re
from collections import Counter, defaultdict

corpus = """
Artificial intelligence is changing the world.
Artificial intelligence is useful in many applications.
Machine learning is a part of artificial intelligence.
Machine learning can solve many problems.
Deep learning is used in artificial intelligence.
The computer learns from data.
The model predicts the next word.
Natural language processing is an important field.
Artificial intelligence can improve education.
Machine learning is used in education.
"""

# Preprocessing
sentences = corpus.lower().split(".")
sentences = [s.strip().split() for s in sentences if s.strip()]

# Create N-gram counts
unigram = Counter()
bigram = Counter()
trigram = Counter()

for words in sentences:
    unigram.update(words)

    for i in range(len(words) - 1):
        bigram[(words[i], words[i + 1])] += 1

    for i in range(len(words) - 2):
        trigram[(words[i], words[i + 1], words[i + 2])] += 1

# Probability functions
def unigram_probability(word):
    return unigram[word] / sum(unigram.values())

def bigram_probability(w1, w2):
    if unigram[w1] == 0:
        return 0
    return bigram[(w1, w2)] / unigram[w1]

def trigram_probability(w1, w2, w3):
    if bigram[(w1, w2)] == 0:
        return 0
    return trigram[(w1, w2, w3)] / bigram[(w1, w2)]
```

```

# Display N-gram information
print("UNIGRAM COUNTS AND PROBABILITIES")
for word, count in unigram.items():
    print(word, count, round(unigram_probability(word), 3))

print("\nBIGRAM COUNTS AND PROBABILITIES")
for pair, count in bigram.items():
    print(pair, count, round(bigram_probability(pair[0], pair[1]), 3))

print("\nTRIGRAM COUNTS AND PROBABILITIES")
for tri, count in trigram.items():
    print(tri, count, round(trigram_probability(tri[0], tri[1], tri[2]), 3))

```

```

# Prediction
def predict(sentence, n):
    words = sentence.lower().split()
    scores = []

    if n == 1:
        for word in unigram:
            scores.append((word, unigram_probability(word)))

    elif n == 2:
        last = words[-1]
        for word in unigram:
            p = bigram_probability(last, word)
            if p > 0:
                scores.append((word, p))

    elif n == 3:
        if len(words) < 2:
            return []

        w1, w2 = words[-2], words[-1]

        for word in unigram:
            p = trigram_probability(w1, w2, word)
            if p > 0:
                scores.append((word, p))

    scores.sort(key=lambda x: x[1], reverse=True)
    return scores[:5]

```

```

# User selects N
n = int(input("\nEnter N (1, 2 or 3): "))

sentence = input("Enter incomplete sentence: ")

print("\nTop-5 predictions:")
result = predict(sentence, n)

if result:
    for word, probability in result:
        print(word, "->", round(probability, 4))
else:
    print("No prediction available.")

```

```

# Unseen N-gram demonstration
print("\nUNSEEN N-GRAM DEMONSTRATION")

print("P(artificial | quantum) =",

```

```

bigram_probability("quantum", "artificial"))

print("P(intelligence | quantum processor) =",
      trigram_probability("quantum", "processor", "intelligence"))

# Accuracy testing
tests = [
    ("artificial intelligence is", "changing"),
    ("machine learning is", "a"),
    ("artificial intelligence can", "improve")
]

correct = 0

for sentence, actual in tests:
    result = predict(sentence, 3)

    predictions = [x[0] for x in result]

    if actual in predictions:
        correct += 1

accuracy = correct / len(tests) * 100

print("\nPrediction Accuracy =", round(accuracy, 2), "%")

```

Limitation

If an N-gram was never seen during training:
This is the **zero-probability problem**.

Task 2 – Backoff and Deleted Interpolation

This task solves the zero-probability problem.

The order is:

Trigram

↓

Bigram

↓

Unigram

The interpolation model combines all three:

with:

$\lambda_1 = 0.2$

$\lambda_2 = 0.3$

$\lambda_3 = 0.5$

```

from collections import Counter
return result[:5]

```

```

# Backoff model
def backoff(sentence):
    words = sentence.lower().split()

    if len(words) < 1:
        return []

    result = []

    for word in unigram:

        if len(words) >= 2:
            p = p_tri(words[-2], words[-1], word)

            if p == 0:
                p = p_bi(words[-1], word)

```

```

else:
    p = p_uni(word)

if p > 0:
    result.append((word, p))

result.sort(key=lambda x: x[1], reverse=True)
return result[:5]

# Deleted interpolation
def interpolation(sentence):
    words = sentence.lower().split()
    result = []

    lambda1 = 0.2
    lambda2 = 0.3
    lambda3 = 0.5

    for word in unigram:

        p1 = p_uni(word)
        p2 = 0
        p3 = 0

        if len(words) >= 1:
            p2 = p_bi(words[-1], word)

        if len(words) >= 2:
            p3 = p_tri(words[-2], words[-1], word)

        probability = (
            lambda1 * p1 +
            lambda2 * p2 +
            lambda3 * p3
        )

        result.append((word, probability))

    result.sort(key=lambda x: x[1], reverse=True)

    return result[:5]

sentence = input("Enter incomplete sentence: ")

print("\nUNSMOOTHED N-GRAM")
for word, p in unsmoothed(sentence):
    print(word, "->", round(p, 4))

print("\nBACKOFF MODEL")
for word, p in backoff(sentence):
    print(word, "->", round(p, 4))

print("\nDELETED INTERPOLATION")
for word, p in interpolation(sentence):
    print(word, "->", round(p, 4))

# Demonstrate unseen trigram
print("\nUNSEEN TRIGRAM")
print("P(machine learning can) =",
      p_tri("machine", "learning", "can"))

```



```
print("P(machine learning quantum) =",
      p_tri("machine", "learning", "quantum"))
```

Comparison

Model	Zero Probability	Coverage	Complexity
Unsmoothed	High	Low	Simple
Backoff	Lower	High	Medium
Interpolation	Very low	Very high	Higher

Why backoff is useful

If:

$P(\text{word} \mid \text{previous two words}) = 0$

the model does not stop. It checks:

Bigram $\rightarrow$ Unigram

Why interpolation is useful

Interpolation does not completely depend on one N-gram. It combines information from:

Unigram + Bigram + Trigram

Therefore it works better with sparse data.

Task 3 – Entropy-Based Evaluation

Entropy measures the uncertainty of a language model.

Higher entropy $\rightarrow$ **more uncertainty**

Lower entropy $\rightarrow$ **more predictable**

```
import math
```

```
total = sum(unigram.values())
```

```
def unigram_prob(word):
    return unigram[word] / total
```

```
def bigram_prob(w1, w2):
    if unigram[w1] == 0:
        return 0
    return bigram[(w1,w2)] / unigram[w1]
```

```
def trigram_prob(w1,w2,w3):
    if bigram[(w1,w2)] == 0:
        return 0
    return trigram[(w1,w2,w3)] / bigram[(w1,w2)]
```

```
# Add-one smoothing
vocab = len(unigram)
```

```
def smooth_prob(w1, w2):
    return (bigram[(w1,w2)] + 1) / (unigram[w1] + vocab)
```

```
def calculate_entropy(sentence, model):
    words = sentence.lower().split()
    probabilities = []
```

```
    for i, word in enumerate(words):
```

```
        if model == "unigram":
            p = unigram_prob(word)
```

```
        elif model == "bigram":
            if i == 0:
                p = unigram_prob(word)
```

```

    else:
        p = bigram_prob(words[i-1], word)

elif model == "trigram":
    if i < 2:
        p = unigram_prob(word)
    else:
        p = trigram_prob(
            words[i-2],
            words[i-1],
            word
        )

elif model == "smooth":
    if i == 0:
        p = unigram_prob(word)
    else:
        p = smooth_prob(words[i-1], word)

if p == 0:
    return float("inf")

probabilities.append(math.log2(p))

entropy = -sum(probabilities) / len(probabilities)

return entropy

```

```
models = ["unigram", "bigram", "trigram", "smooth"]
```

```
for sentence in testing:
```

```
    print("\nSentence:", sentence)
```

```
    for model in models:
```

```
        entropy = calculate_entropy(sentence, model)
```

```
        if entropy == float("inf"):
            print(model, "-> Infinite / very high uncertainty")
        else:
            print(
                model,
                "-> Entropy:",
                round(entropy, 3)
            )

```

Interpretation

For:

The cat sits on the mat

the words are commonly found in the training corpus, so the model can predict them relatively well.

For:

The quantum processor redesigned the

words such as:

quantum

processor

redesigned

are unseen.

Therefore the model gives:

very low probability

and consequently:

high entropy

Entropy interpretation

Entropy	Meaning
Low	Highly predictable
Medium	Moderately predictable
High	Highly uncertain
Infinite	Zero probability occurred

Smoothed models are useful because unseen words/N-grams receive a small non-zero probability instead of zero.

Task 4 – Comparative POS Tagging System

This is the biggest task because it requires three different approaches.

The program below uses the NLTK Brown Corpus.

First install NLTK:

```
pip install nltk
```

Then download Brown Corpus:

```
import nltk
```

```
nltk.download('brown')
```

Complete POS Tagging Program

```
import nltk
```

```
from nltk.corpus import brown
```

```
from collections import Counter, defaultdict
```

```
# Load Brown corpus
```

```
tagged_words = brown.tagged_words()
```

```
# -----
```

```
# TRAINING DATA
```

```
# -----
```

```
word_tag = defaultdict(Counter)
```

```
tag_count = Counter()
```

```
transition = defaultdict(Counter)
```

```
previous_tag = "<START>"
```

```
for word, tag in tagged_words:
```

```
    word = word.lower()
```

```
    word_tag[word][tag] += 1
```

```
    tag_count[tag] += 1
```

```
    transition[previous_tag][tag] += 1
```

```
    previous_tag = tag
```

```
# -----
```

```
# RULE-BASED TAGGER
```

```
# -----
```

```
def rule_based(sentence):
```

```
    words = sentence.lower().split()
```

```
    result = []
```

```
    nouns = ["book", "ticket", "cat", "dog", "student"]
```

```
    pronouns = ["i", "he", "she", "they", "we"]
```

```
    determiners = ["a", "an", "the"]
```

```
    verbs = ["read", "eat", "play", "run"]
```

```
    for word in words:
```

if word in pronouns:

tag = "PRP"

elif word in determiners:

tag = "DT"

elif word in verbs:

tag = "VB"

elif word in nouns:

tag = "NN"

elif word.endswith("ing"):

tag = "VBG"

elif word.endswith("ed"):

tag = "VBD"

elif word.endswith("ly"):

tag = "RB"

elif word.endswith("s"):

tag = "NNS"

else:

tag = "NN"

Example 1

Input:

I book a ticket

Possible output:

RULE-BASED

I/PRP book/VB a/DT ticket/NN

TRANSFORMATION-BASED

I/PRP book/VB a/DT ticket/NN

Here:

book = verb

because:

I book a ticket.

means the action of booking.

Example 2

Input:

I read a book

Here:

read = verb

book = noun

So:

I/PRP read/VB a/DT book/NN

The important point is that the same word can have different POS tags depending on context.

Evaluation Accuracy

For Task 4, you can evaluate each tagger using a test corpus.

The basic accuracy formula is:

A simple evaluation function is:

def accuracy(predicted, actual):

correct = 0

for p, a in zip(predicted, actual):

if p == a:

correct += 1

return correct / len(actual) * 100

For example:

Actual tags:

PRP VB DT NN

Predicted:

PRP VB DT NN

Accuracy = $4/4 \times 100$
= 100%